Lab Work for Week 9: REST API with Django REST Framework (DRF)

Objective

By the end of this lab, students will:

1. Understand the purpose and components of Django REST Framework (DRF).
2. Set up DRF in their Django project.
3. Create API endpoints for listing courses, retrieving course details, and enrolling students.
4. Test the API using tools like Postman or cURL.

Step-by-Step Lab Tasks

---

1. Install Django REST Framework

1. **Task**: Install DRF using pip.

```
pip install djangorestframework
```

2. **Task**: Add rest_framework to INSTALLED_APPS in settings.py:

```
INSTALLED_APPS = [
  ...,
  'rest_framework',
]
```

---

2. Create Serializers

1. **Task**: Create a serializers.py file in the courses app.
    - Add the following code to define serializers for the Course and Lesson models:

```
from rest_framework import serializers
from .models import Course, Lesson

class LessonSerializer(serializers.ModelSerializer):
  class Meta:
    model = Lesson
```

```
        fields = ['id', 'title', 'content', 'video_url']


class CourseSerializer(serializers.ModelSerializer):
    lessons = LessonSerializer(many=True, read_only=True)


    class Meta:
        model = Course
        fields = ['id', 'title', 'description', 'duration', 'lessons']
```

2. **Why This Matters**: Serializers translate model instances into JSON and vice versa.

---

3. Create API Views

1. **Task**: Define API views in views.py using DRF's APIView and Response.
    - Add the following code:

```
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from .models import Course, Lesson
from .serializers import CourseSerializer


class CourseListAPI(APIView):
    def get(self, request):
        courses = Course.objects.all()
        serializer = CourseSerializer(courses, many=True)
        return Response(serializer.data)


class CourseDetailAPI(APIView):
    def get(self, request, pk):
        try:
            course = Course.objects.get(pk=pk)
```

```
    except Course.DoesNotExist:
        return Response({'error': 'Course not found'}, status=status.HTTP_404_NOT_FOUND)
    serializer = CourseSerializer(course)
    return Response(serializer.data)
```

2. **Why This Matters**: These API views handle HTTP requests and return JSON responses.

---

4. Add API Endpoints

1. **Task**: Add URL patterns for the API views in the urls.py file of the courses app.
    - Update urls.py:

```
from django.urls import path
from .views import CourseListAPI, CourseDetailAPI

urlpatterns = [
    path('api/courses/', CourseListAPI.as_view(), name='api_course_list'),
    path('api/courses/<int:pk>/', CourseDetailAPI.as_view(), name='api_course_detail'),
]
```

2. **Why This Matters**: These endpoints allow clients to access the API.

---

5. Test the API

1. **Task**: Start the Django development server.

```
python manage.py runserver
```

2. **Task**: Use Postman or cURL to test the API endpoints:
    - List all courses:
        - URL: http://127.0.0.1:8000/api/courses/
        - Method: GET
    - Get details for a specific course:
        - URL: http://127.0.0.1:8000/api/courses/<course_id>/
        - Method: GET
3. **Task**: Verify the JSON responses.

---

6. Extend the API with Enrollment

1. **Task**: Add a view for enrolling students in a course.
    - Add the following to views.py:

```
from .models import Student


class EnrollStudentAPI(APIView):
  def post(self, request):
    student_email = request.data.get('email')
    course_id = request.data.get('course_id')


    try:
      course = Course.objects.get(id=course_id)
    except Course.DoesNotExist:
      return Response({'error': 'Course not found'}, status=status.HTTP_404_NOT_FOUND)


    student, created = Student.objects.get_or_create(email=student_email)
    student.enrolled_courses.add(course)


    return Response({'message': f'{student.email} has been enrolled in {course.title}'})
```

2. **Task**: Add a URL pattern for the enrollment API.
    - Update urls.py:

```
from .views import EnrollStudentAPI


urlpatterns += [
  path('api/enroll/', EnrollStudentAPI.as_view(), name='api_enroll_student'),
]
```

3. **Test Enrollment**:
    - Use Postman or cURL to send a POST request:

- URL: http://127.0.0.1:8000/api/enroll/
- Body (JSON):

```
{
  "email": "student@example.com",
  "course_id": 1
}
```

Lab Deliverables

1. Code for:
    - serializers.py with CourseSerializer and LessonSerializer.
    - API views in views.py.
    - API endpoints in urls.py.
2. Screenshots of:
    - API response for listing courses.
    - API response for course details.
    - Enrollment API request and response.

Additional Questions for Students

1. How does DRF simplify API development in Django?
2. What is the difference between ModelSerializer and Serializer in DRF?
3. How would you handle additional features like authentication or pagination in the API?